OS / FINAL REFERENCE

ภาพรวม · Process · Thread · Synchronization · CPU

01 OS คือผู้จัดการทรัพยากร

Referee: แบ่ง CPU/Memory/I/O อย่างเป็นธรรม แยกและป้องกันงาน เปิดทางให้สื่อสารกัน · **Illusionist:** สร้างภาพเหมือนแต่ละโปรแกรมมีเครื่องของตนเอง · **Glue:** API/บริการร่วมซ่อนความต่างฮาร์ดแวร์

พัฒนาการ: ไม่มี OS → Batch (คิวงาน) → Multiprogramming (สลับเมื่อรอ I/O) → Time-sharing (แบ่งเวลาเพื่อโต้ตอบ)

ประเมิน: reliability/availability, security, portability, performance (latency/throughput/overhead/fairness), adoption. เป้าหมายมี trade-off; เร็วที่สุดอาจไม่ปลอดภัยหรือย้ายแพลตฟอร์มง่ายที่สุด

02 Process และ Protection

Program = โค้ดที่เก็บไว้; **Process** = instance ที่กำลังทำงาน มี address space + ≥1 thread + ทรัพยากร; **Thread** = execution context (PC, registers, SP, stack)

PCB: PID, state, saved registers/PC/SP, scheduling/accounting, memory mapping, open resources. Code/static data/heap/stack เป็นส่วนของพื้นที่โปรแกรม

User mode: สิทธิ์จำกัด · **Kernel mode:** privileged instructions/ควบคุมฮาร์ดแวร์. Hardware ตรวจ mode และ memory access; timer ช่วยให้ OS ยึด CPU กลับได้

Protection: memory translation + permissions แยก Process และ Kernel; dual mode จำกัดคำสั่ง. โปรแกรมติดต่อ Kernel ผ่าน syscall ที่ตรวจ argument/pointer ก่อนใช้

เหตุเข้า Kernel	ต้นเหตุ
Interrupt	ภายนอก เช่น timer, I/O เสร็จ
Exception	คำสั่งที่รัน เช่น divide-by-zero, page fault
System call	โปรแกรมจงใจขอบริการ OS

03 Kernel mechanism และ Process API

เข้า Kernel: save state → เปลี่ยนไปใช้ stack/สิทธิ์ที่ปลอดภัย → vector ไป handler → ตรวจ/ทำงาน → restore state → return. รายละเอียดลำดับขึ้นกับสถาปัตยกรรม

Kernel stack: ไม่เชื่อ user stack ที่อาจผิด/เป็นอันตราย. Interrupt vector จำกัด entry point. ปิด interrupt ป้องกันการขัดจังหวะบน CPU นั้น ไม่ใช่ lock ทุกคอร์

> **Mode switch = เปลี่ยนระดับสิทธิ์**
> **Context switch = เปลี่ยน thread/process ที่รัน**

Syscall อาจกลับงานเดิมโดยไม่มี context switch. สลับ kernel threads อาจไม่มี mode switch

คำสั่ง	ผลที่ต้องจำ
fork()	สร้าง child; parent ได้ PID ลูก, ลูกได้ 0; ล้มเหลวได้ −1
exec()	แทนโปรแกรมใน Process เดิม; PID เดิม; สำเร็จไม่กลับโค้ดเก่า
wait()	รอลูก/รับสถานะ/เก็บกวาด; อาจกลับทันทีถ้าลูกจบแล้ว
open/read/write/close	I/O ผ่าน file descriptor

Shell: อ่านคำสั่ง → fork → ลูก exec → parent wait (foreground). **Pipe:** ส่ง output เป็น input อีกงาน. IPC มี pipe, shared memory, client–server, files

Monolithic: บริการหลักรวมใน kernel, เรียกกันเร็วแต่ fault กระทบกว้าง · **Microkernel:** core เล็ก บริการหลายส่วนอยู่ user space, isolation ดีแต่มี IPC overhead

04 Concurrency และ Thread Lab

Concurrency: งานดำเนินคาบเกี่ยว (คอร์เดียวสลับได้) · **Parallelism:** รันพร้อมจริงหลายคอร์. Thread ใน Process เดียวแชร์ code/data/heap แต่แต่ละตัวมี PC/registers/stack ของตน

State: Ready → Running → Blocked (รอ I/O/lock) → Ready. หมด quantum: Running → Ready. งานจบ → Terminated

Create: สร้าง thread+stack แล้วเข้าคิว; ไม่รับรองว่าใครเริ่มก่อน · **Yield:** ยอมคืน CPU · **Join:** รอ target จบ · **Exit:** จบ thread · **Sleep:** พักตามเวลา ไม่รับรองว่าอีกงานเสร็จ

Kernel threads: OS จัดคิว สลับผ่าน kernel · **User threads:** library จัดคิว; ผลของ blocking/parallelism ขึ้นกับ mapping. หลาย Thread ไม่ได้แปลว่าเร็วขึ้นเสมอ

Context switch: save registers/SP → เปลี่ยน stack/context → restore → resume. ลำดับ print อาจเปลี่ยนทุกครั้ง; Join ตามลำดับไม่ได้บังคับลำดับการเริ่ม Thread

05 Synchronization และ Deadlock

Race: ความถูกต้องขึ้นกับลำดับเข้าถึงข้อมูลที่ควบคุมไม่พอ · **Critical section:** ส่วนใช้ shared state ที่ต้องควบคุม · **Mutual exclusion:** เข้าได้ทีละตัว · **Atomic:** มองเป็นหนึ่งขั้นที่แบ่งแทรกไม่ได้

Lock: acquire ก่อนใช้ shared state, release เมื่อเสร็จ; ทุกทางเข้าถึงใช้ lock เดียวกัน. **Semaphore:** P/wait รอ count>0 แล้วลดแบบ atomic; V/post เพิ่มและปลุก. ค่าเริ่ม k จำกัดทรัพยากร k หน่วย

CV: wait ปล่อย lock และหลับแบบ atomic; ก่อนกลับต้องได้ lock อีกครั้ง. signal ปลุกหนึ่ง/broadcast ทุกคน แต่ไม่ยก lock ให้ทันที; signal ตอนไม่มีคนรอไม่สะสมเครดิต

```
lock.acquire();
while (!condition) cv.wait(lock);
// access shared state
lock.release();
```

ใช้ **while** เพราะตื่นแล้วเงื่อนไขอาจเปลี่ยน/ตื่นโดยไม่มีเงื่อนไขจริง. ห้ามใช้ Sleep แทน synchronization. Memory barrier จัดลำดับ/การมองเห็น ไม่แทน mutual exclusion

> **Deadlock: M H N C**
> **Mutual exclusion · Hold and wait**
> **No preemption · Circular wait**

Deadlock: รอทรัพยากรกันเป็นวงจรจนคืบหน้าไม่ได้ · **Starvation:** บางงานรอเรื่อยๆ แม้งานอื่นยังเดิน. **Prevention:** ทำลาย ≥1 เงื่อนไข เช่นเรียงลำดับ lock · **Avoidance:** Banker ตรวจ safe state · **Detection/recovery:** ตรวจกราฟแล้ว rollback/ยุติบางงาน. วงจรยืนยัน deadlock เมื่อแต่ละชนิดมี 1 instance

06 CPU Scheduling

นโยบาย	หลัก / ข้อแลกเปลี่ยน
FCFS / FIFO	มาก่อนก่อน; overhead ต่ำ; งานยาวขวางงานสั้น
SJF / SRTF	burst สั้นไม่แย่ง / remaining สั้นแบบแย่ง; งานยาวอาจอด
Round Robin	ผลัด quantum q; q ใหญ่คล้าย FIFO, q เล็กสลับถี่
MLFQ / MFQ	หลายคิว priority; ใช้ครบ slice มักลดชั้น, boost ช่วยกัน starvation

> **TAT = finish − arrival**
> **Waiting = TAT − CPU burst (ไม่มี I/O)**
> **First response = first run − arrival**
> **Throughput = completed jobs / time**

คำเตือนนิยาม: สไลด์ใช้ response time เป็นเวลาจนเสร็จในบางหน้า; อ่านนิยามโจทย์ก่อนแทนสูตร. งานมาถึง/เวลาสลับต้องใส่ใน Gantt ตามที่โจทย์ให้

ตัวอย่าง: A=5, B=2, C=1 มาถึง t=0, ไม่มี overhead. FIFO A→B→C จบ 5,7,8: TAT เฉลี่ย 20/3. SJF C→B→A จบ 1,3,8: TAT เฉลี่ย 4; waiting เฉลี่ย (0+1+3)/3=4/3

05+ ต้นทุนและวิธีประสานงาน

Performance: lock contention, thread overhead, false sharing (คนละข้อมูลแต่ cache line เดียว). ลดด้วย fine-grained lock, per-core data, ownership.
CSP: ส่งข้อความให้เจ้าของข้อมูลจัดการ; ยังออกแบบจน deadlock ได้

OS / FINAL REFERENCE

Address Translation · Virtual Memory · สูตรและวิธีทำ

07 แปลงที่อยู่: ตรวจ → แยก → ค้น → รวม

VA: ที่อยู่ที่โปรแกรมใช้ · **PA:** ที่อยู่ใน RAM · **MMU:** ฮาร์ดแวร์แปลงที่อยู่ · **PTE:** mapping + permissions/present/dirty/use · **TLB:** cache ผลการแปลง ไม่ใช่ cache เนื้อหาข้อมูล

Base & Bounds: สมมติ bound คือความยาว; ตรวจ 0 ≤ VA < bound แล้ว PA=base+VA. **Segmentation:** VA=(segment,offset); table ให้ base/length/permission แล้ว PA=base+offset เมื่อ offset<length และสิทธิ์ผ่าน

Segmentation: ขนาดต่างกัน จัดตาม code/data/stack แชร์/ป้องกันได้ แต่ external fragmentation. **Paging:** ขนาดเท่ากัน จัด frame ง่าย ไม่มี external fragmentation สำหรับการจัด frame แต่มี internal fragmentation ในหน้าสุดท้าย

```
Page size S = 2^b bytes → offset = b bits
VPN = floor(VA / S) · offset = VA mod S
PTE[VPN] → PFN
PA = PFN × S + offset
```

จำ: "เปลี่ยนเลขหน้า เก็บ offset เดิม" • VPN ใช้ lookup ห้ามนำ VPN ไปคำนวณ PA แทน PFN • offset เท่าขนาดหน้าไม่ได้ • เช็ก present และ permission ก่อนใช้ PA

ขนาดตาราง: byte-addressable, ตารางระดับเดียวเต็มช่วง: VA n bits, page 2^b → VPN n−b bits → entries=$2^{(n-b)}$. Table bytes=entries×PTE bytes. PA m bits → PFN m−b bits.

```
32-bit VA, 4 KiB page → b=12, VPN=20
Entries=2²⁰=1,048,576
PTE 4 bytes → table=4 MiB / process
```

KiB=2^{10} bytes; MiB=2^{20} bytes; 1 hex digit=4 bits. คำว่า KB/MB ในโจทย์ให้ตรวจว่าหมายถึงฐาน2หรือฐาน10

EX ตัวอย่าง Paging + Segment

VA=0x1234=4660; S=1024; VPN4→PFN9

1. offset bits = $\log_2 1024 = 10$
2. VPN=⌊4660/1024⌋=4
3. offset=4660−4096=564=0x234
4. PA=9×1024+564=9780=**0x2634**

Base=0x4000, length=0x800: offset0x234 ผ่าน → PA0x4234. offset0x800 ไม่ผ่าน (ต้อง < length ไม่ใช่ ≤)

2L Multilevel และ TLB

แบ่ง VA=[outer index | inner index | offset]. ใช้ outer หา table ระดับ2 แล้ว inner หา PTE→PFN; ค่อยรวม offset. จัด table เฉพาะส่วนที่ใช้ ลดพื้นที่ sparse address space แต่มี lookup มากขึ้น

```
32-bit, page4KiB, split 10|10|12
VA 0x12345678 → outer=72 (0x48)
inner=837 (0x345), offset=1656 (0x678)
ถ้า PFN=0xABC → PA=0xABC678
```

TLB hit: ได้ translation เร็ว (ยังตรวจสิทธิ์) · **TLB miss:** เดิน page table; หน้าอาจอยู่ RAM แล้ว · **Page fault:** ระบบต้องจัดการ mapping/presence/permission ก่อนคำสั่งไปต่อ

```
EAT = h(t+m) + (1−h)[t+(k+1)m]
t=TLB lookup, m=RAM access
k=จำนวนระดับ page table; ไม่มี page fault
```

สมมติ lookup เป็นลำดับ ไม่ซ้อนกัน ไม่คิด cache อื่น: h=.9, t=10ns, m=100ns, k=2 → .9×110+.1×310=**130ns**. ถ้าโจทย์ให้ model อื่น ให้ใช้ model นั้น

08 Virtual Memory และ Page Fault

Demand paging: ให้เฉพาะหน้าที่ต้องใช้อยู่ RAM; หน้าที่เหลืออาจอยู่ file/swap หรือยังไม่ได้จัดสรรจริง. VA ใหญ่กว่า RAM ได้ ไม่ได้ทำให้ RAM เพิ่มและอาจช้าจาก I/O

Locality: temporal=ใช้สิ่งเดิมซ้ำเร็วๆ; spatial=ใช้ที่อยู่ใกล้กัน. Cache ใช้ความสัมพันธ์นี้ลดเวลาเฉลี่ย. **Write-through:** ส่งการเขียนลงชั้นถัดไปทันที; **write-back:** เก็บ dirty แล้วเขียนเมื่อจำเป็น

```
Page fault → trap → ตรวจว่า access ถูกต้อง
หา/จัดสรร frame (อาจ evict)
โหลดจาก file/swap หรือ zero-fill
update PTE/TLB → restart คำสั่งเดิม
```

ถ้า address ผิด/permission ไม่ผ่าน อาจยุติหรือส่ง signal แทนโหลดข้อมูล. ไม่ใช่ทุก fault ต้องอ่านดิสก์ เช่น zero-fill และ copy-on-write

Eviction: เลือก victim → ยกเลิก mapping/PTE และ TLB ที่เกี่ยวข้อง → ถ้า dirty และต้องเก็บข้อมูลให้ write-back → ใช้ frame ใหม่. **Dirty:** ถูกเขียน · **Use/reference:** ถูกเข้าถึง

Memory-mapped file: map file เป็น address range แล้วใช้ load/store; หน้าที่ขาด fault เข้ามาได้ แชร์และลดการ copy แต่ยังมีต้นทุน fault/I/O

RP เลือกหน้าออกเมื่อ frame เต็ม

วิธี	เลือก victim
FIFO	เข้ามานานที่สุด; hit ไม่เปลี่ยนลำดับเข้า
MIN / OPT	จะถูกใช้อีกไกลที่สุดในอนาคต; optimal benchmark
LRU	ไม่ได้ใช้นานที่สุดในอดีต; hit ต้องอัปเดต recency
LFU	ใช้น้อยครั้งที่สุด; ต้องกำหนด tie-break/aging
Clock	วนเข็ม: R=1 ลดเป็น0และให้โอกาส; R=0 เลือกออก

Belady's anomaly: เพิ่ม frame แล้ว fault กลับมากขึ้นได้ใน FIFO; LRU/MIN แบบมาตรฐานไม่มี anomaly นี้. **Thrashing:** working set เกิน RAM → fault ถี่ → เสียเวลาสลับหน้า

วิธีทำโจทย์: เขียน reference ทีละช่อง → ระบุ hit/fault → ถ้าว่างเติม → ถ้าเต็มเลือก victim ตาม policy → อัปเดตลำดับ/บิต → รวม fault. เริ่ม frame ว่างถ้าโจทย์กำหนด

```
Reference: 1,2,3,1,4,2 · 3 frames ว่าง
FIFO: F F F H F H → 4 faults
LRU: F F F H F F → 5 faults
```

FIFO anomaly ตัวอย่าง: 1,2,3,4,1,2,5,1,2,3,4,5 → 3 frames=9 faults, 4 frames=10 faults. มาก frame ไม่รับประกัน FIFO ดีกว่า

EAT Page fault มีต้นทุนสูง

```
EAT = (1−p) × normal + p × faultTotal
p=โอกาส fault ต่อ reference
faultTotal รวมบริการ fault และ access ที่เสร็จแล้ว
```

normal=100ns, faultTotal=8ms=8,000,000ns, p=10^{-6} → EAT=99.9999+8=**107.9999ns**. ถ้า fault time ไม่รวม access หลัง retry ต้องบวกตามสมมติฐานโจทย์

หน่วย: 1ms=10^6ns, 1µs=10^3ns. ห้ามเอา ms บวก ns โดยไม่แปลง; hit rate กับ fault rate ไม่ใช่ตัวเดียวกัน

✓ ก่อนส่งคำตอบ

ปรนัย: หา keyword เช่น always, shared, atomic, preemptive; ตรวจตัวเลือกกับเงื่อนไขโจทย์ ไม่ใช้ความรู้สึก

อธิบาย: นิยาม → ปัญหา/เหตุ → กลไกเป็นลำดับ → ผล/ข้อแลกเปลี่ยน → ตัวอย่าง. ระบุว่าอะไรแชร์/แยกและใครรอใคร

คำนวณ: เขียน units + assumptions → สูตร → แทนค่า → ตรวจขอบเขต/สิทธิ์ → คำตอบทั้งฐานสิบ/hexถ้าต้องการ. Offset คงเดิมเมื่อ paging